DSA

LAB 7

Name: Muzammil Ahmed

Roll No: CT-24307

Section: F

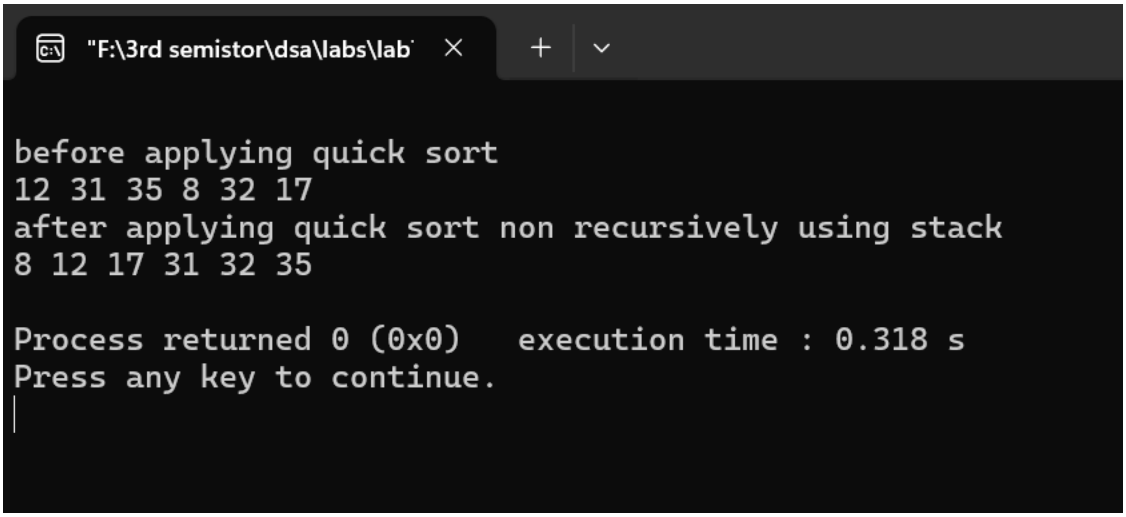
Q1:

Code:

```cpp
#include<iostream>
#include<vector>
#include<stack>
using namespace std;
int partition(vector<int>& arr, int st, int end) {
int idx = st - 1, pivot = arr[end];
for (int j=st; j<end; j++) {
if (arr[j]<=pivot) {
idx++;
swap(arr[j], arr[idx]);
}}
idx++;
swap(arr[end], arr[idx]);
return idx;}
void quickSortIterative(vector<int>& arr, int st, int end) {
stack<int> s;
s.push(st);
s.push(end);
while(!s.empty()){
end =s.top();
s.pop();
st=s.top();
s.pop();
if(st<end){
int middle=partition(arr, st, end);
if(middle+1< end){
s.push(middle+1);
s.push(end);
}
```

```
30      if(st<(middle-1)){
31      s.push(st);
32      s.push(middle-1);
33      }}}}
34      int main() {
35      vector<int> arr = {12, 31, 35, 8, 32, 17};
36      cout <<"\nbefore applying quick sort"<< endl;
37      for(int val : arr) {
38      cout<< val<<" ";
39      }
40      quickSortIterative(arr, 0, arr.size() - 1);
41      cout<< "\nafter applying quick sort non recursively using stack"<< endl;
42      for(int val:arr) {
43      cout <<val<< " ";
44      }
45      cout << endl;
46      }
47
```

Output:



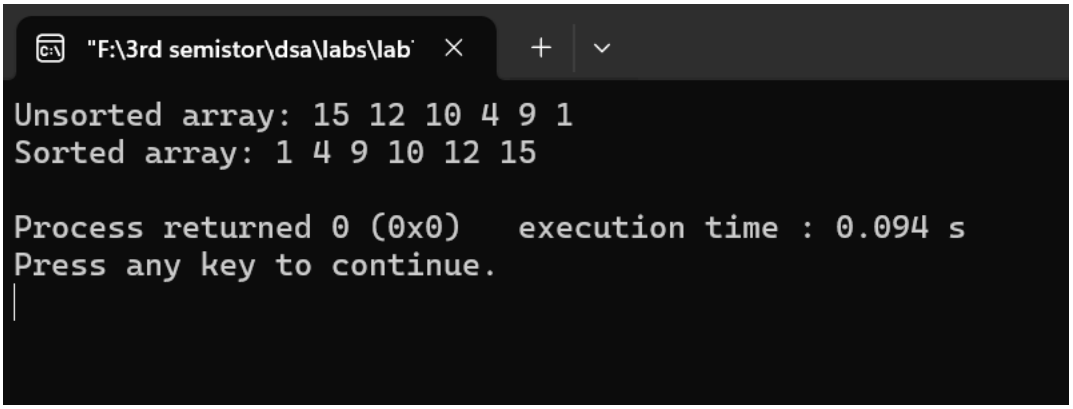
Q2:

Code:

```
#include<iostream>
using namespace std;
void merge(int arr[],int left,int mid,int right){
int n1=mid-left+1;
int n2=right-mid;
int L[n1];
int R[n2];
for(int i=0;i<n1;i++){
L[i]=arr[left+i];
}
for(int j=0;j<n2;j++){
R[j]=arr[mid+ j+ 1];
}
int i=0;
int j=0;
int k=left;
while(i<n1 && j<n2){
if(L[i]<=R[j]){
arr[k]=L[i];
i++;
}
else{
arr[k]=R[j];
j++;
}
k++;
}
while(i<n1){
arr[k]=L[i];
```

```
30      i++;
31      k++;
32      }
33      while(j<n2){
34      arr[k]=R[j];
35      j++;
36      k++;}}
37      void mergesort(int arr[],int n){
38      for(int size=1; size<n; size*=2){
39      for(int left=0; left<n-1; left+=2*size){
40      int mid=min(left+size-1,n-1);
41      int right=min(left+2*size-1,n-1);
42      merge(arr,left,mid,right);
43      }}}
44      void printArray(int arr[], int size) {
45      for (int i = 0; i < size; i++)
46      cout << arr[i] << " ";
47      cout << endl;
48      }
49      int main(){
50      int arr[] = {15, 12, 10, 4, 9, 1};
51      int n=sizeof(arr)/sizeof(arr[0]);
52      cout << "Unsorted array: ";
53      printArray(arr, n);
54      mergesort(arr,n);
55      cout << "Sorted array: ";
56      printArray(arr,n);
57      }
58
59
```

Output:



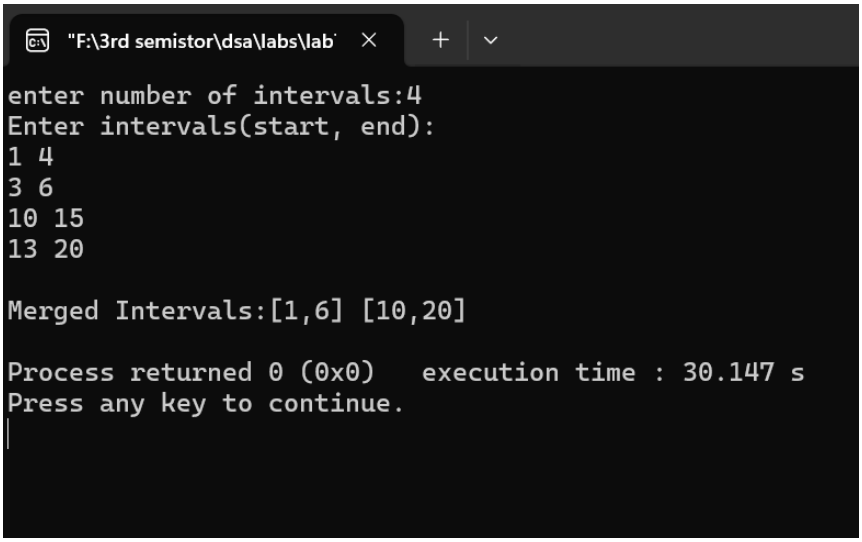
Q3:

Code:

here | q1part2.cpp | q2.cpp | q3.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main(){
int n;
cout<<"enter number of intervals:";
cin >> n;
vector<vector<int>> intervals(n, vector<int>(2));
cout<<"Enter intervals(start, end):"<<endl;
for(int i = 0; i < n; i++){
cin>>intervals[i][0]>>intervals[i][1];
}
sort(intervals.begin(), intervals.end());
vector<vector<int>> merge;
for (int i = 0; i < n; i++){
int start = intervals[i][0];
int end = intervals[i][1];
if (merge.empty()){
merge.push_back({start, end});
}else if(start <= merge.back()[1]) {
if(end > merge.back()[1]){
merge.back()[1] = end;
}}
else{
merge.push_back({start, end});
}
}
cout<<"\nMerged Intervals:";
for(int j = 0; j < merge.size(); j++) {
cout<<"["<<merge[j][0]<<","<<merge[j][1]<<"] ";
}
cout<<endl;
return 0;
}
```

Output:



Q4:

```cpp
#include <iostream>
using namespace std;
void mergeArrays(int arr[], int left, int mid, int right){
int n1 = mid - left + 1, n2 = right - mid;
int leftArr[1000], rightArr[1000];
for(int i = 0; i < n1; i++) leftArr[i]=arr[left + i];
for(int j = 0; j < n2; j++)rightArr[j]=arr[mid + 1 + j];
int i = 0, j = 0, k = left;
while(i < n1 && j < n2){
if(leftArr[i] <= rightArr[j])arr[k++]=leftArr[i++];
else
arr[k++]=rightArr[j++];
}
while(i<n1)arr[k++]=leftArr[i++];
while(j<n2)arr[k++]=rightArr[j++];
}
void mergeSort(int arr[], int left, int right){
if(left < right){
int mid=(left + right)/2;
mergeSort(arr, left, mid);
mergeSort(arr, mid + 1, right);
mergeArrays(arr, left, mid, right);
}
}
int longestHarmoniousSubsequence(int arr[], int n){
mergeSort(arr, 0, n - 1);
int maxLength = 0;
int i=0;
while(i < n){
```

```
30  int currentValue=arr[i];
31  int countCurrent=0;
32  while(i < n && arr[i] == currentValue){
33  countCurrent++;
34  i++;
35  }
36  int j=i;
37  int nextValue=currentValue + 1;
38  int countNext = 0;
39  while(j < n && arr[j]==nextValue){
40  countNext++;
41  j++;
42  }
43  if(countNext > 0 && countCurrent + countNext > maxLength)
44  maxLength=countCurrent + countNext;
45  }
46  return maxLength;
47  }
48  int main(){
49  int arr[]={1,3 , 2, 2, 6, 2, 4, 8};
50  int n = sizeof(arr) / sizeof(arr[0]);
51  cout <<"Original Array:";
52  for(int i = 0; i < n; i++)
53  cout <<arr[i]<< " ";
54  cout<<endl;
55  int result = longestHarmoniousSubsequence(arr, n);
56  cout<<"Longest Harmonious Subsequence Length:"<<result<<endl;
57  cout<<"Sorted Array: ";
58  for(int i = 0; i < n; i++)
59  cout<<arr[i]<<" ";
60  cout<<endl;
61  return 0;
62  }
63
```

OUTPUT:

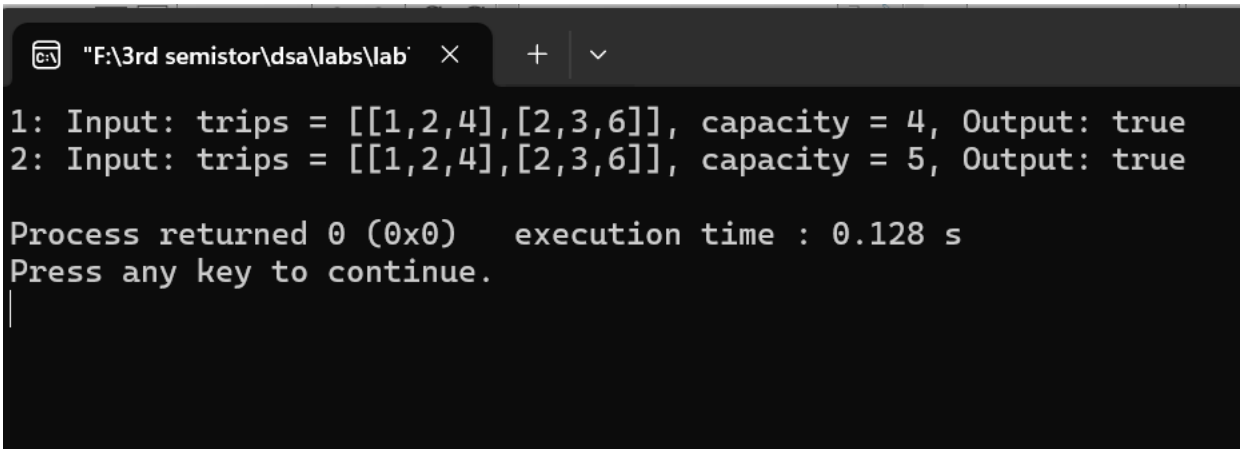
```
"F:\3rd semistor\dsa\labs\lab

Original Array:1 3 2 2 6 2 4 8
Longest Harmonious Subsequence Length:4
Sorted Array: 1 2 2 2 3 4 6 8

Process returned 0 (0x0)   execution time : 0.183 s
Press any key to continue.
```

Q5:

```cpp
#include<iostream>
using namespace std;
struct Stop {
int location;
int change;
};
void swapStops(Stop &a, Stop &b){
Stop temp =a;
a=b;
b=temp;
}
int partition(Stop arr[], int low, int high){
int pivotValue=arr[high].location;
int i=low-1;
for(int j=low; j<high; j++){
if (arr[j].location <= pivotValue){
i++;
swapStops(arr[i], arr[j]);
}
}
swapStops(arr[i + 1], arr[high]);
return i + 1;
}
void quickSort(Stop arr[], int low, int high){
if(low < high){
int pivotIndex=partition(arr, low, high);
quickSort(arr, low, pivotIndex - 1);
quickSort(arr, pivotIndex + 1, high);
}
```

```
}
bool canCarPool(int trips[][3], int totalTrips, int capacity){
int totalStops = totalTrips * 2;
Stop *events = new Stop[totalStops];
int index = 0;
for(int i = 0; i < totalTrips; i++){
int numPassengers = trips[i][0];
int startPoint=trips[i][1];
int endPoint=trips[i][2];
events[index++]={startPoint, numPassengers};
events[index++]={endPoint, -numPassengers};
}
quickSort(events,0,totalStops-1);
int currentPassengers=0;
for(int i = 0; i < totalStops; i++){
currentPassengers+=events[i].change;
if(currentPassengers > capacity){
delete[] events;
return false;
}}
delete[] events;
return true;
}
int main(){
int trips1[2][3]={{1, 2, 4}, {2, 3, 6}};
int capacity1=4;
cout<< "1: Input: trips = [[1,2,4],[2,3,6]], capacity = 4, Output: ";
cout<<(canCarPool(trips1, 2, capacity1) ? "true" : "false")<<endl;
int trips2[2][3]={{1, 2, 4}, {2, 3, 6}};
int capacity2=5;
cout<<"2: Input: trips = [[1,2,4],[2,3,6]], capacity = 5, Output: ";
cout<<(canCarPool(trips2, 2, capacity2) ? "true" : "false")<<endl;
return 0;
}
```

OUTPUT:


"F:\3rd semistor\dsa\labs\lab
1: Input: trips = [[1,2,4],[2,3,6]], capacity = 4, Output: true
2: Input: trips = [[1,2,4],[2,3,6]], capacity = 5, Output: true
Process returned 0 (0x0)   execution time : 0.128 s
Press any key to continue.